# Lista de Exercícios: Testes com Cypress e PHPUnit

Sugestão de Projeto: Doce Gestão (confeitaria em Laravel)

---

Rotas, campos e seletores citados são sugestões; adapte ao código do projeto. Ambiente: ./docker/install.sh e ./docker/start.sh (http://localhost:8000). PHPUnit: php artisan test.

## Parte 1: Cypress (1 a 20)

### Básico

**1. Configuração do Cypress.** Instale o Cypress no projeto (npm install cypress --save-dev), abra-o com npx cypress open e defina o baseUrl como http://localhost:8000 no arquivo de configuração.

**2. Visitar a página inicial.** Crie o arquivo cypress/e2e/home.cy.js com um teste que visita a página inicial e confirma que o texto "Doce Gestão" aparece (cy.contains).

**3. Navegação pelo menu.** Clique nos links de Produtos, Clientes, Pedidos e Fretes e valide a URL de cada página com cy.url().should('include', ...).

**4. Listagem de produtos.** Visite a listagem de produtos e verifique que a tabela existe, que tem ao menos uma linha (dados do seed) e que os cabeçalhos principais aparecem (nome, categoria, preço, estoque).

**5. Abrir o formulário de novo produto.** Na listagem, clique no botão de criar produto e verifique a URL e a presença dos campos: nome, categoria (select), preço, estoque e estoque mínimo.

**6. Seletores estáveis e organização.** Adicione atributos data-cy nas views Blade e refatore os testes anteriores para usá-los. Agrupe tudo com describe/it e use beforeEach para o cy.visit.

### Intermediário

**7. Cadastro de produto.** Preencha nome, categoria, preço e estoque, envie o formulário e valide a mensagem de sucesso ou o redirecionamento para a listagem.

**8. Validação de formulário vazio.** Envie o formulário de produto sem preencher nada e verifique as mensagens de erro de cada campo obrigatório.

**9. Valores inválidos.** Teste preço negativo, estoque negativo e texto em campos numéricos. Cada cenário deve ser um it separado.

**10. Edição de produto.** Abra a edição de um produto existente, altere o preço e salve.

**11. Exclusão de produto.** Cadastre um produto de teste, exclua-o (trate o confirm do navegador, se houver) e valide o resultado.

**12. CRUD de clientes.** Crie, edite e exclua um cliente com contato e endereço, em um único fluxo de teste ou em testes separados.

**13. Alerta de estoque baixo.** Cadastre um produto com estoque abaixo do mínimo e verifique o alerta visual (classe, cor ou badge). Depois cadastre outro acima do mínimo e verifique que o alerta não aparece.

**14. Fixtures de dados.** Crie cypress/fixtures/produto.json e cliente.json e use cy.fixture() para preencher os formulários em vez de dados escritos no teste.

**Avançado**

**15. Comandos customizados.** Em cypress/support/commands.js, crie cy.criarProduto(dados) e cy.criarCliente(dados) e reaproveite-os nos testes dos exercícios seguintes.

**16. Paginação.** Garanta que existam mais registros que o limite da página (use o comando do exercício 15), clique em "próxima" e valide que o conteúdo mudou e que a página anterior continua acessível.

**17. Fluxo completo de pedido.** Crie um cliente e um produto, registre um pedido escolhendo ambos nos selects e valide o status. Leia o estoque antes e depois do pedido e confirme a baixa automática na quantidade correta.

**18. Estoque insuficiente.** Tente criar um pedido com quantidade maior que o estoque disponível.

**19. Fluxo de frete.** Crie um frete vinculado a um pedido (endereço, taxa, data prevista e rastreio) e atualize o status até a entrega, validando a listagem a cada etapa.

**20. Interceptação e teste ponta a ponta.** Use cy.intercept para monitorar o POST de criação de um pedido e validar o status HTTP. Crie uma task do Cypress que

execute php artisan migrate:fresh --seed (via docker compose exec) antes da suíte e escreva um teste E2E: cliente, produto, pedido e frete.

## Parte 2: PHPUnit (21 a 30, nível básico)

**21. Primeiro teste**. Execute php artisan test, leia a saída e crie um teste simples em tests/Unit usando assertTrue(true). Em seguida, altere para assertFalse(true) e observe a falha.

**22. Rota inicial.** Em tests/Feature, crie um teste que faz $this->get('/') e valida assertStatus(200).

**23. Rotas dos módulos.** Verifique que as listagens de produtos, clientes, pedidos e fretes respondem com status 200 (ajuste ao nome real das rotas).

**24. Conteúdo da página.** Use assertSee para conferir que a listagem de produtos exibe um texto esperado, como um título ou o nome de um produto criado no teste.

**25. Rota inexistente.** Faça um GET em uma rota que não existe e valide o status 404 com assertNotFound() ou assertStatus(404).

**26. Banco de dados com RefreshDatabase**. Use o trait RefreshDatabase, crie um produto (factory ou Model::create) e valide com assertDatabaseHas.

**27. Cadastro via POST.** Envie um POST válido para criar um cliente com $this->post(), verifique o redirecionamento (assertRedirect) e confirme o registro no banco.

**28. Validação de dados.** Envie um POST de produto sem o nome e verifique o erro de validação com assertSessionHasErrors. Faça o mesmo com preço inválido.

**29. Atualização e exclusão.** Crie um produto, atualize o preço com PUT/PATCH e valide com assertDatabaseHas. Crie um cliente, exclua com DELETE e valide com assertDatabaseMissing.

**30. Baixa de estoque.** Crie um produto com estoque 10 e registre um pedido de 3 unidades. Verifique que o estoque ficou em 7. Em outro teste, peça mais que o estoque e confirme que o pedido não foi criado e que há erro de validação.